

ASTRAQ

Autonomous Spatial Tracking & Alignment for Optical Links



Technical Report

Design, methods, test methodology and measured performance · Final version

SMART INDIA HACKATHON 2026 · SIH26169

AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile FSOC Terminals
ISRO / Department of Space · Software · Smart Automation, Space Technology

Contents

Summary	3
Problem understanding	3
System architecture	4
Tracking methods	5
AI method: the learned beacon verifier	7
Video benchmark (camera bypass)	8
Test methodology	9
Performance analysis	9
Limitations	11
Future improvements	11
References	11

About the numbers

Every number in this report was produced by running the delivered code (commands in Section 7). Raw outputs are in docs/test-results/. Where the prototype does not meet a requirement, or where something was not tested, this report says so.

SECTION 1

Summary

ASTRAQ is a software testbed for the **coarse alignment** stage of a mobile free-space optical communication (FSOC) terminal, built for SIH26169. A simulated pan/tilt camera on a transportable ground station must find a satellite's beacon in a noisy image, centre it and keep it centred, under the disturbances listed in the problem statement (PS). The same tracking pipeline can also run on recorded video with the camera bypassed.

PS requirement	Target	Measured (default scenario unless stated)	
Acquisition time	≤ 2 s	1.13 s mean, 1.50 s max (20 random starts)	✓
Tracking error	≤ 10 px	2.97 px RMS	✓
Target loss	< 5 %	0 % after lock (8 of 9 scenarios); ≤ 1.5 % with 30 % random dropouts	✓
Re-acquisition	≤ 1 s	0.03 s max over 40 outages (Occlusion scenario)	✓
Processing speed	≥ 20 FPS	30 Hz loop; detector ≈ 1.5 ms/frame; video 280 fps (640×480), 30–32 fps (2000×2000)	✓
Video centroid error	minimise	0.057 px (640×480); 0.08–0.12 px (2000×2000, 1 % salt & pepper)	✓
Performance log / report	automatic	per-frame CSV, JSON, HTML/Markdown report	✓
Executable	required	Electron desktop app (Windows .exe built; Linux build tested)	●

Main technical contributions: (1) wide-field acquisition followed by narrow tracking; (2) a Kalman filter in azimuth/elevation with rate feed-forward, which cut error from 14.9 px to 3.7 px; (3) a trained neural-network **beacon verifier** that raised per-frame detection from 89.9 % to 98.7 % and cut false detections from 8.4 % to 0.59 %; (4) prediction-following re-acquisition; (5) a coarse-to-fine detector for 2000×2000 video.

SECTION 2

Problem understanding

An optical link has a beam divergence of tens of microradians, so the two terminals must point at each other far more precisely than any GPS/IMU prediction can. Real terminals therefore use a pointing, acquisition and tracking (PAT) chain: a **coarse** stage (camera + gimbal) brings the partner's beacon into the field of a **fine** stage (fast steering mirror + quad cell). The PS asks for the coarse stage, in simulation, with a virtual camera and measurable performance.

Parameters taken from the PS

Quantity	Value	Consequence (derived)
Image	640 × 480, ≥ 30 Hz	one frame every 33 ms
Field of view	4° × 3°	0.00625°/px; 10 px = 0.0625°
Gimbal rate	5–10 °/s	a 12.5° raster with a 4° FOV takes ≈ 11 s at 5 °/s

Quantity	Value	Consequence (derived)
Beacon spot	5–20 px	a known-size bright blob: sub-pixel centroiding is possible
Noise	Gaussian $\sigma \leq 20$, salt & pepper $\leq 10\%$, Poisson	impulse repair + robust (median/MAD) statistics needed
Jitter	± 20 px/frame	white jitter alone gives ≈ 16 px RMS: a floor for any coarse stage
Targets	≤ 2 s acquisition, ≤ 10 px, $< 5\%$ loss, ≤ 1 s re-acq., ≥ 20 FPS	scored against ground truth

Two derived facts shaped the design. First, with only the 4° optics a random start inside a 12.5° field cannot always be found within 2 s, so ASTRAQ searches with a 12° wide field and switches to 4° once the beacon is confirmed (as NASA's OPALS terminal did). Second, the ± 20 px jitter case has a physical floor above 10 px; we report it rather than hide it.

Interpretations we had to choose

- **Pointing error** is measured as the distance between the true beacon position and the image centre (what the link cares about); the centroid error (detected vs true) is logged separately.
- **Loss** is the share of frames after first lock with the tracker not in TRACKING/LOCKED, counted only while the beacon is visible, so a scheduled cloud outage is not a tracking failure.
- **Re-acquisition** time starts when the beacon becomes visible again and ends at the next confirmed track.

SECTION 3

System architecture

Browser: Web Worker @ 30 Hz · Python: FastAPI + WebSocket (same algorithms) · Desktop: Electron

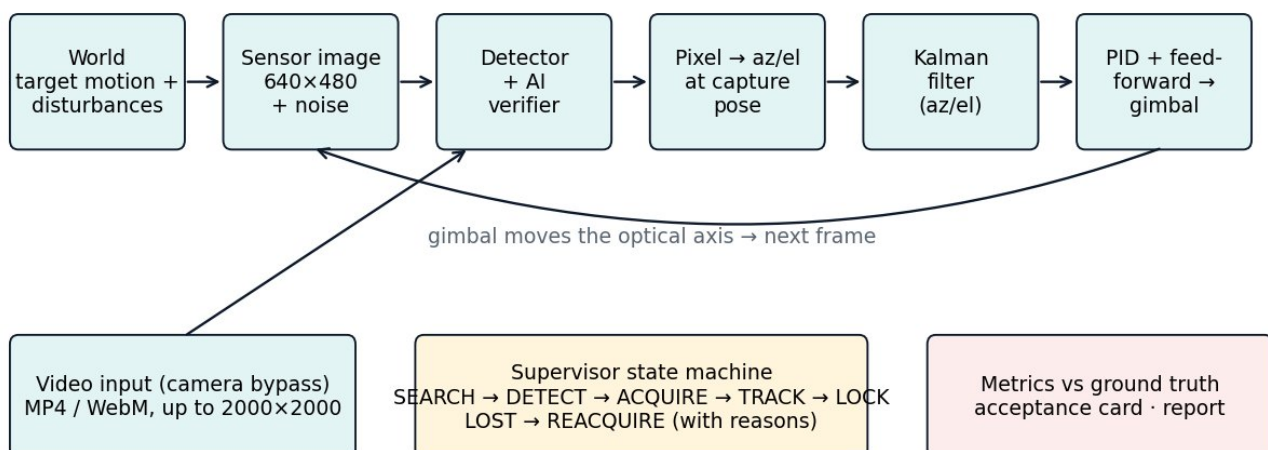


Figure 1. The closed loop run every 1/30 s, and the video path that bypasses the camera.

The simulation core is pure TypeScript with no UI dependencies. It runs in a browser Web Worker (default; no server needed), is mirrored in a numpy port behind FastAPI (REST + WebSocket), and is packaged with Electron as a desktop app. The UI (React, three.js) only

consumes telemetry snapshots through a TelemetryProvider interface (local worker, remote WebSocket or replay), so the same HUD works with every engine. Only the metrics module sees ground truth; the tracker never does.

Module	Responsibility
optics/camera, sensor	pinhole model with PS intrinsics; renders the 8-bit frame: stars, beacon (square/disk/Gaussian), atmosphere, turbulence, noise
disturbance	vibration, jitter, platform motion, wind torque, dropouts, scheduled occlusion, decoy
trajectory	9 motion models incl. custom waypoints and a LEO pass with ephemeris error
detection	impulse repair, robust background, blob extraction, pyramid for large frames, AI verifier, sub-pixel centroid
estimation	constant-velocity Kalman filter on az/el with χ^2 gate and adaptive R
control	PID with 3 anti-windup guards, rate feed-forward, gimbal with rate/accel limits and 40 ms lag, spiral search
supervisor	8-state machine with reasons; confirmation (2 of 3), lock (6 frames < 10 px), coasting, re-acquisition
analysis	metrics, acceptance card, recorder, batch runner, performance report, link budget (simplified)
video	frame analyzer with pixel Kalman, automatic spot size, truth comparison, CSV log

SECTION 4

Tracking methods

4.1 Detection

- **Impulse repair:** a pixel that is the extreme of its 8-neighbourhood by a wide margin is replaced by the neighbourhood median (removes salt & pepper without blurring the beacon).
- **Background:** median and MAD over a sampled grid give a noise estimate σ that is not biased by the beacon or stars.
- **Segmentation:** threshold at background + $k\sigma$ ($k = 5$; when the verifier is on, $\min(k, 4\sigma)$ so dimmer candidates reach it), 8-connected components.
- **Ranking:** every component is ranked by $\text{SNR} \cdot \sqrt{\text{area}}$; features are computed for the strongest 64 (a beacon is never lost behind noise blobs in raster order).
- **Gating:** while tracking, candidates farther than 60 px from the prediction are ignored (ROI).
- **Centroid:** intensity-weighted mean over background-subtracted pixels: ≈ 0.05 px RMS in clear conditions, < 1.5 px at the PS noise maxima.
- **Large frames** (> 1.5 Mpx): impulse repair, then decimation for the search, then the centroid is refined at full resolution around the coarse hit.

4.2 Estimation

The detected pixel is converted to an absolute direction using the gimbal pose *at capture time*, so the filter models the target's real motion rather than the motion caused by the gimbal itself. Each axis is a constant-velocity Kalman filter ($q = 2 \text{ (}^\circ/\text{s}^2\text{)}^2/\text{Hz}$, $r = 1.5 \text{ px}$) with a χ^2 innovation gate (25) and a measurement noise that adapts to the recent innovation, which keeps jitter from locking the gate out. After a loss the filter is re-initialised with the last rate estimate kept.

4.3 Control

A PID ($K_p 8$, $K_i 1$, $K_d 0.05$) acts on the angular error and adds the Kalman rate as feed-forward. Windup is prevented by an integral clamp, conditional integration while the rate saturates, and integral separation (integrate only near the set-point). The gimbal limits rate to $5 \text{ }^\circ/\text{s}$ and acceleration to $30 \text{ }^\circ/\text{s}^2$, with a 40 ms first-order lag. The ablation below shows why the estimator matters:

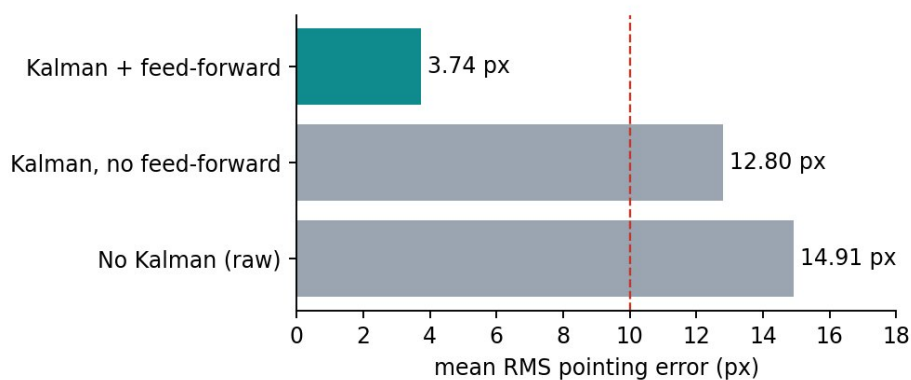


Figure 2. Kalman/feed-forward ablation (Python engine, 3 trajectories $\times$ 3 seeds, 15 s).

4.4 Acquisition and supervision

SEARCHING runs a square spiral over the $\pm 6.25^\circ$ uncertainty field with the wide 12° field of view. A candidate becomes DETECTED, and is confirmed after 2 of 3 spatially consistent detections (ACQUIRING); the optics switch to 4° and the gimbal slews. TRACKING becomes LOCKED after 6 consecutive frames under 10 px. Up to 5 missed frames are coasted on the prediction; more gives LOST. REACQUIRING follows the prediction for 1 s (outages are often short), then spirals locally, and returns to SEARCHING after 3 s. Every transition carries a human-readable reason.

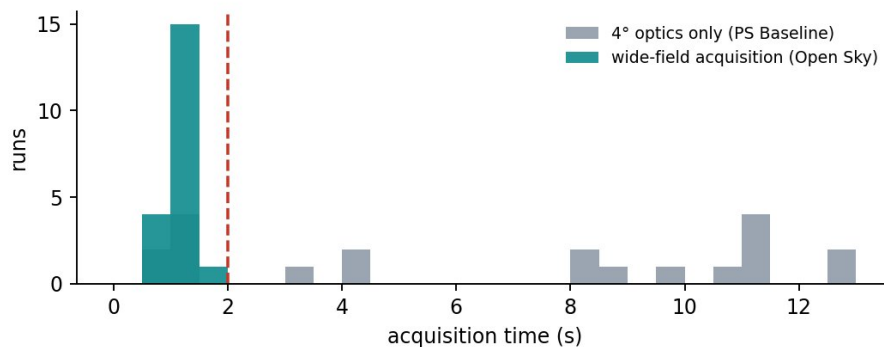


Figure 3. Wide-field acquisition versus 4° optics only (20 random starts each).

SECTION 5

AI method: the learned beacon verifier

The hard part of acquisition is not locating a spot but deciding *which* spot is the beacon when stars, noise clusters, glints and fog compete with it. The v1.0 detector scored candidates with a hand-tuned formula. v1.1 replaces that score with a small neural network trained for the job.

Item	Choice
Input	11 features per candidate: log area ratio to expected spot, log SNR, log fill ratio, log aspect , flatness, saturation, log expected size, ring contrast, log blur σ , contrast, spread
Model	MLP 11 $\rightarrow$ 16 $\rightarrow$ 8 $\rightarrow$ 1, tanh hidden units, sigmoid output ($\approx$ 340 parameters, $<$ 0.05 ms per candidate)
Training data	frames rendered by the simulator across all atmospheres, σ 0-20, salt & pepper 0-10 %, Poisson, turbulence, spot 5-20 px, 3 spot shapes, 4° and 12° FOV, decoys and stars. Label 1 = within max(3 px, $\frac{1}{2}$ spot) of the true beacon
Training	Adam, 60 epochs, batch 512, learning rate $3 \cdot 10^{-3}$ with step decay, weight decay 10^{-4} (numpy, web/scripts/verifier/train.py)
Operating point	accept $p \geq 0.7$
Deployment	weights as JSON; identical inference in TypeScript and Python, checked by a shared fixture test

Offline evaluation on 3,879 held-out frames (109,230 candidates): candidate ROC AUC 0.9817 $\rightarrow$ **0.9997**; frames with the beacon detected 89.9 % $\rightarrow$ **98.7 %**; frames with a wrong spot accepted 8.40 % $\rightarrow$ **0.59 %**. The worst condition (fog, wide FOV) improved from 63 % to 89 % detection.

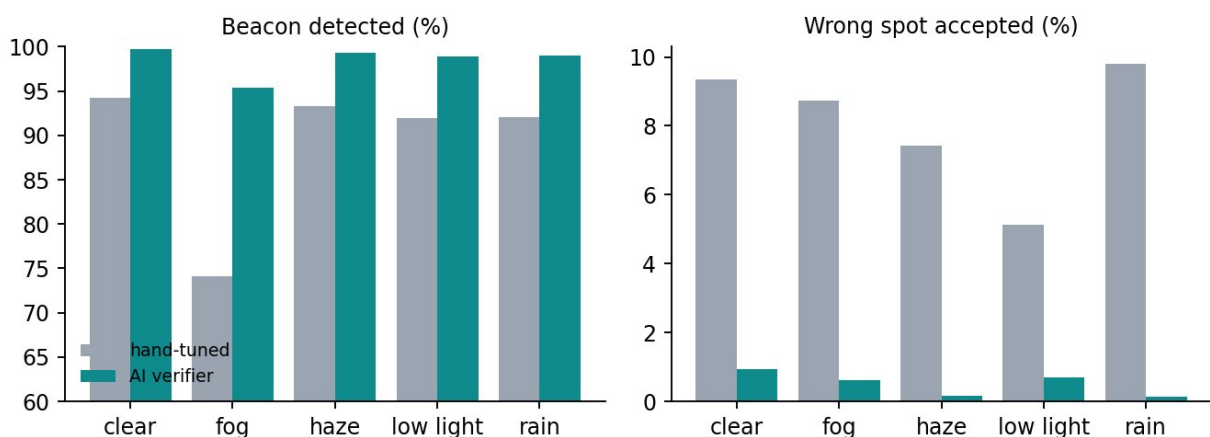


Figure 4. Offline evaluation by atmosphere (frame-weighted over FOV and decoy groups).

Closed-loop effect: with every other setting unchanged, the verifier fixed the Weak Beacon scenario and changed nothing in the others (identical results in 8 presets).

Weak Beacon scenario, closed loop, 20 seeds × 15 s

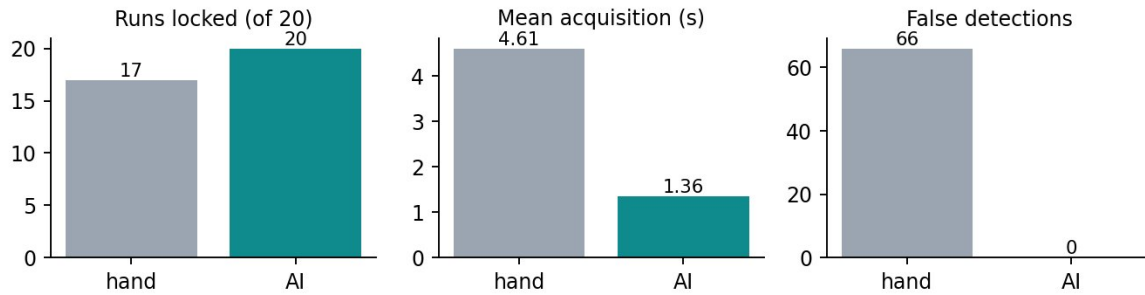


Figure 5. Closed-loop ablation, TypeScript engine.

Design choice and limitation

Why a verifier and not YOLO: the graded quantity is centroid accuracy, and an intensity-weighted centroid (≈ 0.05 px) is far more precise than box regression. The network therefore decides *which* blob, and classical centroiding measures *where*. The verifier has only been trained on simulated frames; retraining on real footage is the next step.

SECTION 6

Video benchmark (camera bypass)

The PS's second benchmark replaces the camera with a 30 fps video that covers a full screen. ASTRAQ decodes the video in the browser (requestVideoFrameCallback, with an optional frame-exact seek mode) or on the server (OpenCV), and feeds each frame to the same detector and verifier. Because there is no gimbal, tracking uses a pixel-space Kalman filter whose process noise and gate scale with frame size ($W/640$) and with any skipped frames. The spot size is estimated automatically from the strongest blobs if not given. Output: per-frame centroid CSV, and a report with detection rate, acquisition, lock retention, re-acquisition, centroid RMSE against an optional truth CSV, and processing speed.

Video	Engine	Acquisition	Centroid RMSE	Re-acq.	Speed
640×480, $\sigma = 8$, MP4	Python	0.03 s	0.057 px	—	280 fps
2000×2000, 1 % s&p, 0.6 s outage, MP4	Python	0.20 s	0.121 px	0.47 s	32 fps
Built-in 2000×2000 stream (same disturbances)	Browser	0.20 s	0.082 px	0.47 s	30 fps

Lock retention on the outage videos is 92 %: the missing 8 % is the 0.6 s during which the beacon is not in the video plus the re-acquisition interval.

SECTION 7

Test methodology

- **Unit and integration tests:** 67 TypeScript (vitest) and 36 Python (pytest + httpx), covering geometry, camera model, detector accuracy at the PS noise maxima, Kalman, PID/gimbal limits, state machine, closed loop on 7 trajectories, occlusion re-acquisition, verifier, large-frame detection, video analyzer, reports, REST and WebSocket API, and the orbit maths of the visual space traffic. All 103 pass.
- **Monte-Carlo batch:** `python scripts/run_batch.py <preset> 20 15: 9 presets × 20 seeds (1000-1019) × 15 s`, image-based detection, verifier on.
- **Ablations:** Kalman/feed-forward (`scripts/ablation.py`); verifier offline (`train.py/evaluate.py`) and closed loop (`closed-loop.ts`).
- **Video:** synthetic videos with exact ground truth from `make_test_video.py` (sizes, noise, salt & pepper, outages) and the built-in browser stream.
- **UI:** headless Chromium screenshots of every view, theme, panel and state; production-build console check; clean install from the delivered ZIP.
- **Ground truth** is known exactly in simulation; the tracker never reads it.

SECTION 8

Performance analysis

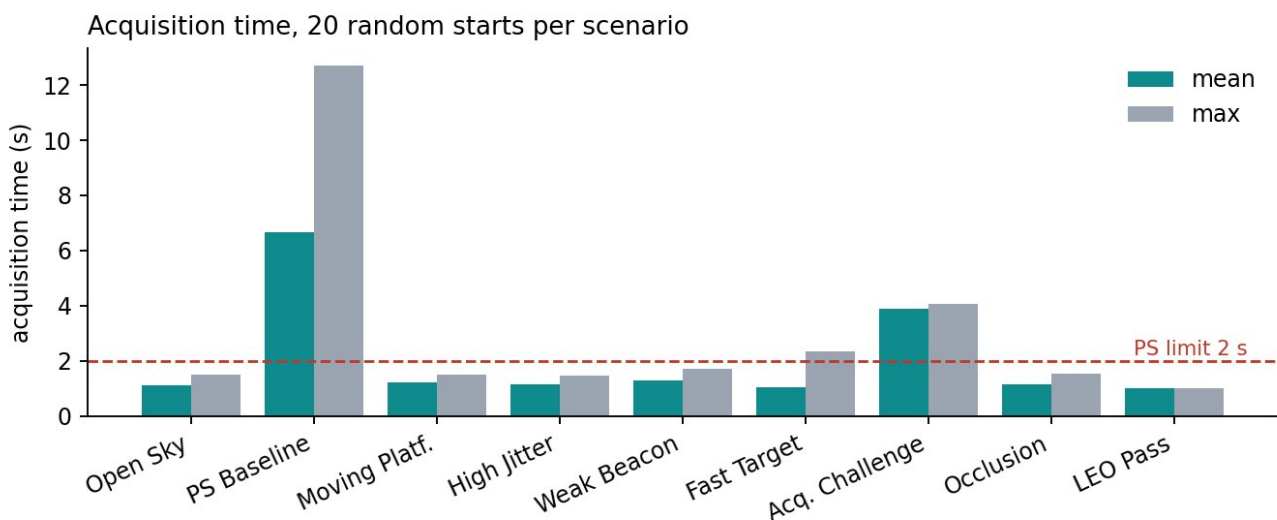


Figure 6. Acquisition time per scenario.

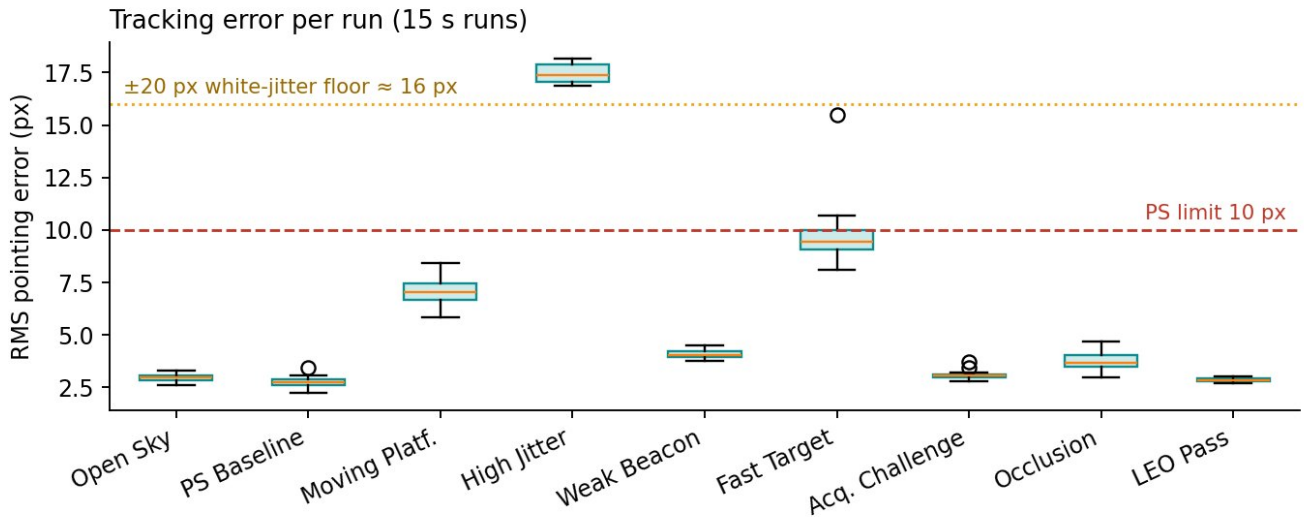


Figure 7. RMS pointing error per run.

Scenario	Locked	Acq. mean / max	RMS	Re-acq. (n, max)
Open Sky	20/20	1.13 / 1.50 s	2.97 px	—
PS Baseline (4° only)	20/20	6.67 / 12.70 s	2.78 px	—
Moving Platform	20/20	1.20 / 1.50 s	7.02 px	—
High Jitter (±20 px/frame)	20/20	1.13 / 1.47 s	17.44 px	—
Weak Beacon	20/20	1.29 / 1.70 s	4.09 px	8, 0.17 s
Fast Target	20/20	1.03 / 2.33 s	9.79 px	—
Acquisition Challenge	20/20	3.88 / 4.07 s	3.08 px	14, 0.13 s
Occlusion & Reacquisition	20/20	1.16 / 1.53 s	3.73 px	40, 0.03 s
LEO Pass	20/20	0.99 / 1.00 s	2.86 px	—

Where the requirements are not met

- **PS Baseline acquisition** (4° optics only): 6.7 s mean. This is a geometric limit of scanning 12.5° with a 4° field at 5 °/s; the wide-field mode meets 2 s.
- **High Jitter**: 17.4 px. Uniform ±20 px white jitter alone is ≈ 16 px RMS and reverses every frame, faster than a 30 Hz coarse loop can react; this belongs to the fine stage.
- **Fast Target**: mean 9.8 px, with some runs above 10 px (worst 15.5 px): a 1.5 °/s random target near the limits of a rate-limited gimbal.
- **Acquisition Challenge**: 3.9 s, because the beacon starts in the far corner with 30 % dropouts and a decoy.

Computational cost

Detection takes about 1.5 ms per 640×480 frame in the browser and about 15 ms in Python; the simulation runs at 30 Hz, and the FastAPI engine streamed at 32 fps over WebSocket. Rendering was tested only with software WebGL, where frame rates are not representative of a GPU.

SECTION 9

Limitations

- Simulation only: no real camera, gimbal or optical bench.
- The AI verifier is trained on simulated frames and has not seen a real sensor.
- The fine-pointing stage is represented only by a hand-over criterion.
- Link budget and acquisition probability are simplified models (labelled in the UI).
- Earth rotation during a run is ignored; terrain colours and city lights are procedural (coastlines are real data).
- The Windows executable was built but not run on Windows; it is not code-signed.
- The space traffic (other satellites, ISS) is a visual layer only, and deep-sky positions are approximate.
- Headless test browsers lack H.264, so in-browser MP4 upload was verified with WebM; the server decodes MP4.

SECTION 10

Future improvements

- Retrain the verifier on real footage (an LED filmed at night, then the real acquisition camera) and add hard-negative mining.
- An IMM filter (constant velocity / acceleration / turn) for manoeuvring targets such as Fast Target.
- A shake-state estimator using IMU data to separate followable from unfollowable platform motion.
- A fine-pointing stage (fast steering mirror + quad cell) to close the jitter gap.
- Hardware in the loop through the two existing seams: the frame source and the pan/tilt rate command.
- Signed installer and automatic updates for the desktop app.

SECTION 11

References

- [1] Y. Kaymak et al., "A Survey on Acquisition, Tracking, and Pointing Mechanisms for Mobile Free-Space Optical Communications", IEEE Communications Surveys & Tutorials, 2018.
- [2] NASA JPL, OPALS (Optical Payload for Lasercomm Science) mission descriptions, 2014–2015.
- [3] Cailabs, TILBA-TOGS Transportable Optical Ground Station, <https://www.cailabs.com/aerospace-defense/laser-communications/transportable-optical-ground-station/>
- [4] DLR, Transportable Optical Ground Station (TOGS), <https://elib.dlr.de/78398/1/8517-5.pdf>
- [5] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems", J. Basic Engineering, 1960.
- [6] H. Lim et al., "Centroid Error Analysis of Beacon Tracking under Atmospheric Turbulence", Remote Sensing 13(10):1931, 2021.
- [7] "Star Centroiding Based on Fast Gaussian Fitting", Sensors 18(9):2836, 2018.
- [8] "Disturbance Feedforward Control for EOTS LOS Stabilization on a Moving Platform", Sensors 18(12):4350, 2018.
- [9] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.
- [10] Natural Earth public-domain map data; three.js; React Three Fiber; FastAPI; Electron.